Chair of Cognitive Systems
TUM School of Computation, Information and Technology
Technical University of Munich

Welding Robot

Report for the final project of the module
Multi-Sensory-Based-Robot-Dynamic-Manipulation

You-Ri Su ✉, Jose Antonio Botello Vazquez ✉, Pablo Eusse Martinez ✉, and Tobias Töws ✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ you-ri.su@tum.de
✉ antonio.botello@tum.de
✉ ge43ruw@mytum.de
✉ tobias.toews@tum.de

February 19, 2026

Abstract —
This project presents the development of an autonomous welding robot system integrating perception, planning, and control within a ROS-based framework. A planar homography-based perception pipeline is combined with camera-separated calibration to obtain metrically consistent weld trajectories. The control architecture includes joint-space fallback control, operational-space tracking with singularity monitoring, and an admittance-based force regulation layer for compliant interaction. Experimental validation demonstrates stable trajectory tracking, reliable singularity avoidance, and robust force adaptation during contact. The resulting system illustrates the integration of multi-sensory perception and dynamic manipulation in a cohesive robotic framework.

1 Introduction

This project presents the development and implementation of a complete vision-based robotic welding trajectory tracking system using a UR10 manipulator. The system integrates computer vision, coordinate frame calibration, trajectory generation, operational space control, and admittance control into a unified architecture.

The main objective is to enable the robot to:

- detect a weld seam using a phone camera,
- convert the detected seam into a metrically consistent trajectory,
- transform the trajectory into the robot reference frame,
- track the trajectory using operational space control,
- maintain compliant contact using admittance control,

The system operates fully autonomously using ROS, OpenCV, and a model-based controller.

2 System Overview

The system consists of three main subsystems:

1. Vision subsystem
2. Trajectory generation subsystem
3. Robot control subsystem

The architecture is shown below:

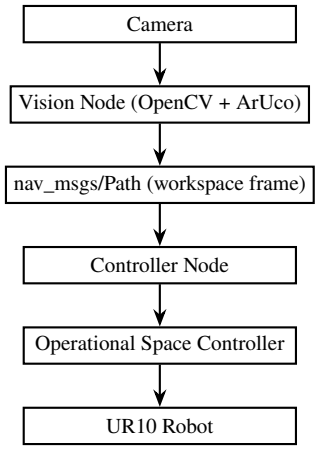


Figure 1 System architecture showing vision-to-control pipeline

3 Vision System

The vision system estimates the workspace geometry, the robot base pose, and the weld trajectory from a monocular RGB camera stream. The overall objective is to transform pixel-level detections into metrically consistent trajectory points expressed in the robot base coordinate frame. In RViz, the global fixed frame is set to `world`.

3.1 Camera Calibration and Workspace Definition

ArUco markers are square fiducial markers widely used in computer vision for pose estimation and camera calibration. Each marker contains a unique binary pattern that allows robust identification and precise localization in an image. By detecting the marker corners, it is possible to estimate their position and orientation relative to the camera.

Four ArUco markers define the workspace plane.

Marker	ID	Purpose
Top-left	0	Workspace corner
Top-right	1	Workspace corner
Bottom-left	2	Workspace corner
Bottom-right	3	Workspace corner

Table 1 ArUco markers used for workspace calibration

Using these markers, a homography is computed:

$$H_{\text{img}\rightarrow\text{workspace}} \tag{1}$$

This allows transformation from image coordinates to real-world coordinates and ensures metric accuracy.

3.2 Weld Line Detection

The weld seam is detected using color segmentation.

The processing steps are:

1. Convert image from BGR to HSV:

$$HSV = \texttt{cv::cvtColor}(BGR) \tag{2}$$

2. Apply color threshold:

$$mask = \texttt{cv::inRange}(HSV, lower, upper) \tag{3}$$

3. Apply morphological filtering:
   - Opening
   - Closing

   to remove noise and improve segmentation quality.

3.3 Skeleton Extraction

The weld line is reduced to a one-pixel wide skeleton using Zhang-Suen thinning:

$$skeleton = \texttt{thinning}(mask) \tag{4}$$

Advantages of skeletonization include:

- Accurate path center extraction
- Minimal noise sensitivity
- Robust path representation

3.4 Path Extraction

Skeleton pixels are traced to create an ordered path.

The tracing algorithm performs the following steps:

- Detect endpoints
- Follow connected pixels
- Generate ordered trajectory points

This produces the trajectory:

$$P = \{p_1, p_2, \ldots, p_n\} \tag{5}$$

3.5 Metric Conversion

Using homography transformation:

$$P_{\text{workspace}} = H \cdot P_{\text{image}} \tag{6}$$

The trajectory is published as:

`nav_msgs/Path`

in the coordinate frame:

`workspace_frame`

All trajectory points lie on the calibrated workspace plane:

$$z = 0 \tag{7}$$

The transformation between the workspace frame and the robot base frame was determined through

manual measurement. The relative position of the workspace origin with respect to the robot base was measured physically and implemented as a fixed offset in the controller.

This approach assumes a static experimental setup and avoids the need for automatic pose estimation while providing sufficient accuracy for trajectory tracking.

4 Trajectory Generation

The controller receives the weld trajectory from the vision system in the form of a ROS message:

`nav_msgs/Path weld_path`

This message contains a sequence of discrete trajectory points defined in the workspace frame. To enable smooth robot motion, these discrete points are converted into a continuous trajectory using linear interpolation.

The desired position is computed as:

$$\mathbf{x}_d = \mathbf{p}_i + \alpha \left(\mathbf{p}_{i+1} - \mathbf{p}_i \right) \tag{8}$$

where:

- $\mathbf{p}_i$ and $\mathbf{p}_{i+1}$ are consecutive trajectory points
- $\alpha \in [0, 1]$ is the interpolation parameter
- $\mathbf{x}_d$ is the desired interpolated position

The desired velocity is computed as:

$$\dot{\mathbf{x}}_d = \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{T} \tag{9}$$

where T represents the time interval between trajectory points.

This interpolation ensures smooth and continuous robot motion during trajectory tracking.

5 Control Module

5.1 Joint Controller

The joint-space controller is used as a safe and robust fallback mode, both for the initial motion towards a non-singular configuration (from which Cartesian control can take over) and for returning from Cartesian tracking. In our implementation, the controller is formulated as a velocity-based tracking law with a sliding variable. Given desired joint position/velocity/acceleration references $(q_d, \dot{q}_d, \ddot{q}_d)$, the joint error is defined as $e_q = q - q_d$. A proportional term is injected into the reference velocity,

$$\dot{q}_r = \dot{q}_d - K_p e_q,$$

and the corresponding sliding variable becomes $s_q = \dot{q} - \dot{q}_r$. The commanded torques are then chosen as a dissipative feedback

$$\tau = -K_{\text{reg}}\, s_q,$$

where $K_{\text{reg}} \in \mathbb{R}^{6\times6}$ is a diagonal regularization (damping) matrix. This structure is inspired by passivity-based designs and can be extended to a full computed-torque/adaptive controller by adding a model-based feedforward term. In our code base, the model-based terms were taken from the tutors provided UR10 dynamics model. Also the baseline joint controller intentionally keeps the torque law simple to prioritize stability.

5.2 Cartesian Controller

For trajectory execution, the controller switches to an operational-space (Cartesian) formulation in which the translational end-effector motion is regulated directly. The desired Cartesian trajectory is generated, same as for joint control, by the state machine with timed phases: (i) a linear motion towards the start of the track, and (ii) trajectory tracking phases. During this motion the controller monitors manipulability using the Yoshikawa measure $w = \sqrt{\det(JJ^\top)}$; when the manipulability indicates a near-singular configuration, the control space is switched back to joint-space operation to avoid numerical issues in the Jacobian inversion. The operational-space formulation follows standard approaches in robot dynamics and control [1].

The operational-space controller uses the translational part of the Jacobian. Given joint state $(q, \dot{q})$, the full Jacobian $J \in \mathbb{R}^{6\times6}$ is obtained from the UR10 model and transformed into the world frame; the translational block is then extracted to map joint velocities to end-effector linear velocity. The end-effector linear velocity is computed as $\dot{x} = J_t(q)\dot{q}$, where $J_t \in \mathbb{R}^{3\times3}$ denotes the translational Jacobian of the first three joints. Note that the full UR10 translational Jacobian is $J_t^{full} \in \mathbb{R}^{3\times6}$, and here we intentionally work with a 3-DoF reduced model for translational motion.

In Cartesian space, we define the position and velocity errors as $e_x = x - x_d$ and $e_{\dot{x}} = \dot{x} - \dot{x}_d$. A PD

tracking term is implemented by defining a reference Cartesian velocity and Acceleration

$$\dot{x}_r = \dot{x}_{des} - K_p e_x,$$

$$\ddot{x}_r = \ddot{x}_{des} - K_d e_{\dot{x}},$$

and mapping it to a reference joint velocity via $\dot{q}_r = J_t^{\dagger}\dot{x}_r$ and a joint acceleration $\ddot{q_r} = J_t^{\dagger}(\ddot{X}_r - \dot{J}^{\dagger}\dot{q}_r)$
. The corresponding sliding variable is $s = \dot{q} - \dot{q}_r$.

$$\tau = -K_{\text{cart}}\, s \; + \; \tau_{\text{ff}}.$$

Here, K_{cart} is a (diagonal) gain matrix, and τ_{ff} denotes a model-based feedforward component computed from the robot dynamics. Since the reference values in joint space are vectors of $\mathbb{R}^3$ the missing three values are filled with zeros.

$$\tau_{\text{ff}} = M(q)\ddot{q}_r + C(q,\dot{q})\dot{q}_r + g(q),$$

In our implementation, the end-effector orientation is stabilized only through joint 4 (wrist 1) and joint 5 (wrist 2). For joint 4 we use a geometric alignment term computed as

$$\text{dot} = z^{\top} R_{w,5}^{-1}(q)\, x,$$

where $x = [1,0,0]^{\top}$ and $z = [0,0,1]^{\top}$, and $R_{w,5}^{-1}(q) \in \mathbb{R}^{3\times 3}$ is the rotation block extracted from $T_{w,5}$. The corresponding torque is then applied as a proportional feedback

$$\tau_4 = -\,\text{dot} \cdot \mathit{gain},$$

with *gain* set to 50 in our experiments. Joint 5 is additionally biased towards a nominal wrist angle using $\tau_5 = 10\left(\frac{\pi}{2} - q_5\right)$.

5.3 Integration of Admittance Layer

The admittance controller is integrated directly on top of the operational-space controller. Instead of modifying the torque law, the admittance model generates a Cartesian offset x_a that adapts the desired trajectory online.

The modified reference becomes

$$x_d^* = x_d + x_a, \qquad \dot{x}_d^* = \dot{x}_d + \dot{x}_a.$$

These updated references are used within the same operational-space tracking law. This hierarchical structure preserves the stability properties of the Cartesian controller while enabling compliant interaction in the contact direction.

5.4 Force Controller

Compliant interaction is achieved through a Cartesian admittance formulation driven by the measured external wrench F_{ext}. The end effector is modeled as a mass in a gravitational field with an added damping term.

$\ddot{x}_a = -G + (F_{ext} - D\dot{x})/M$

where G denotes a constant downward force, M the virtual mass, F_{ext} the force vector of the sensor rotated into world frame and D denotes the damping.

The resulting offset x_a modifies the desired Cartesian trajectory as described in Section 5.2. This formulation enables force regulation along the contact normal direction while maintaining accurate trajectory tracking in the tangential directions.

6 Experiments and Results

6.1 Experimental Setup

The proposed system was evaluated using a UR10 industrial robot equipped with a camera-based vision system. The camera was positioned above the workspace to observe the weld seam and detect the trajectory using the vision pipeline described previously.

The workspace was defined using ArUco markers to establish a metric reference frame. The transformation between the workspace frame and the robot base frame was determined through manual measurement and implemented as a fixed offset in the controller.

The robot was controlled using an operational space controller to track the generated trajectory. All components were integrated using ROS, enabling real-time communication between the vision system and the robot controller.

6.2 Trajectory Detection Results

The vision system successfully detected the weld seam and generated a continuous trajectory. The skeletonization process produced a clean one-pixel-wide representation of the weld line, allowing accurate path extraction.

Figure 2 shows the setup with the camera, the UR10 robot and the trajectory detector.



Figure 2 Experimental setup and extracted trajectory

6.3 Robot Trajectory Tracking

The robot successfully tracked the generated trajectory using operational space control. The controller ensured smooth motion and stable tracking throughout the trajectory.

Figure 3 shows the robot during trajectory execution.

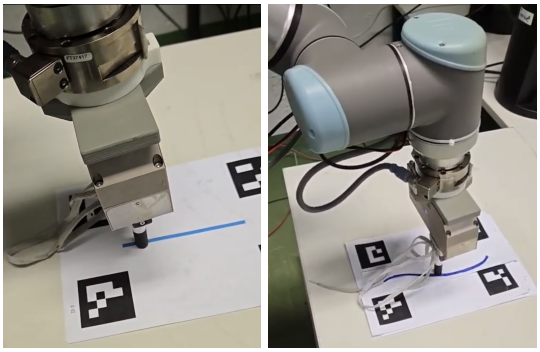


Figure 3 Vision detection and robot trajectory tracking

6.4 System Performance

The system operated in real time and demonstrated stable performance. The main observations include:

- Reliable weld seam detection using the vision system
- Accurate trajectory generation in metric coordinates
- Stable and smooth robot trajectory tracking
- Successful integration of vision and control components

The results demonstrate that the proposed system is capable of autonomous weld trajectory detection and execution using vision-based control.

7 Conclusion

This project presented a complete vision-based weld trajectory detection and tracking system using a UR10 robotic manipulator. The system successfully integrates computer vision, trajectory generation, and operational space control into a unified framework.

The vision subsystem was able to reliably detect the weld seam and generate a metrically consistent trajectory using image processing and homography transformation. The controller converted this trajectory into a continuous motion and successfully tracked it using operational space control.

Experimental results demonstrated stable and smooth trajectory execution, confirming the effectiveness of the proposed approach. The integration of ROS enabled real-time communication between the vision system and the robot controller, ensuring reliable system operation.

Overall, the system demonstrates the feasibility of autonomous weld trajectory detection and execution using vision-based control, providing a flexible and robust solution for robotic automation tasks.

8 Future Work

Future improvements could focus on automatic calibration between the camera, workspace, and robot coordinate frames to improve accuracy and flexibility. This would eliminate the need for manual measurements and allow the system to adapt to changes in the environment.

Additionally, a more precise approach would be to estimate the pose of the robot base frame automatically instead of defining it manually. This could be achieved using fiducial markers, such as ArUco markers, attached to the robot base and detected by the vision system. This would enable automatic computation of the transformation between the workspace frame and the robot base frame,

improving accuracy, repeatability, and ease of system setup.

Further improvements could also include smoother trajectory generation using spline interpolation and the integration of force feedback for enhanced interaction control during welding tasks.

References

[1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.